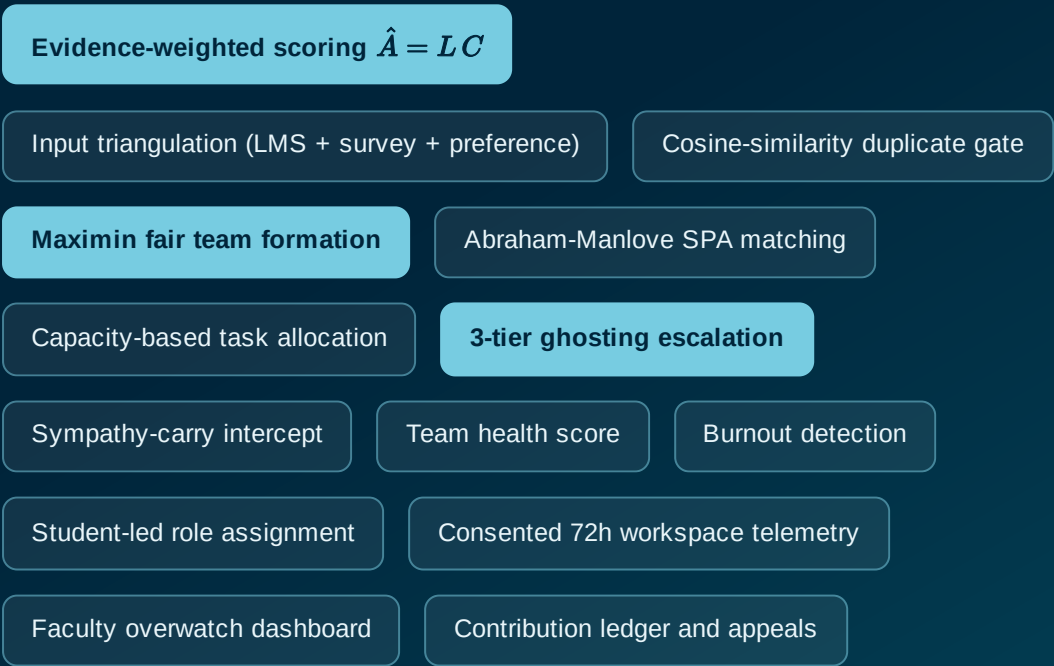


AEGIS

Automated Engineering Group Intelligence System

A capstone allocation and governance engine that checks skills against real evidence instead of trusting a self-report, forms teams that are balanced rather than stacked, watches for the two ways teams quietly fail, and hands lecturers decisions instead of raw logs. Everything it watches, students are told about on day one.

WHAT THE SYSTEM IS BUILT FROM



Governance and policy basis. AEGIS runs under a written policy stack built for Sri Lanka: the Personal Data Protection Act No. 9 of 2022, the Computer Crimes Act No. 24 of 2007, and University Grants Commission rules. It keeps consent at onboarding, role-based access, data minimisation, a published appeals route, and human review before any automated decision affects a student.

TEAM	TRACK	DELIVERABLE
The Amateurs	Scenario 03	Case Analysis

Zaidh Najeed (Group Leader) · Aasim Niyas · Irusha Chathurya Pettanayaka
Informatics Institute of Technology

Zaidh Najeeb
GROUP LEADER

BSc Computer Science
zaidhnajeeb98@gmail.com
+94 72 321 9159

Aasim Niyas
MEMBER

BSc Artificial Intelligence & Data
Science
mohaasim111@gmail.com
+94 76 560 9118

Irusha Chathurya Pettanayaka
MEMBER

BSc Artificial Intelligence & Data
Science
icpettanayaka@gmail.com
+94 71 702 7000

All members: Informatics Institute of Technology, Colombo, Sri Lanka

SECTION ONE

01. Problem Understanding

Capstone allocation looks like a scheduling chore until you run it at the scale of a full cohort. Then it turns into four separate problems wearing one coat. We spent our first session pulling them apart, because a fix for one of them usually does nothing for the other three, and a few of the obvious fixes quietly make a different problem worse.

The four ways allocation fails

The first failure is honest-looking but wrong input. Students rate their own skills, and those ratings drift upward. Some of it is ordinary optimism. A lot of it is the Dunning-Kruger effect, where the people with the least experience in a skill are the most confident about it. If the matching engine trusts those numbers, it builds teams on data that was already broken before any algorithm touched it.

The second failure shows up later, when proposals come in. Across a large cohort you get groups submitting projects that are structurally the same idea with the serial numbers filed off. Nobody copied anyone. They just landed on the same obvious thing. Left alone it dents the academic value of the work and makes grading awkward.

The third failure is the one that does the real damage, and it is social rather than technical. Two patterns recur. A strong student quietly absorbs a teammate's work to protect the grade, which we refer to as the sympathy carry, and it ends in burnout while hiding the fact that someone else never contributed. The mirror image is ghosting, where a member goes dark and the gap only surfaces near a deadline. Both are invisible to a lecturer reading status updates, because the people involved have every reason not to report it.

The fourth failure is administrative. A supervisor handling many teams cannot sit and read raw activity logs for each one. Even if every signal needed were sitting in a database, the time cost of parsing it by hand means problems get noticed after they have already cost someone a grade.

Why these have to be solved together. Verified skills are pointless if a balanced team still collapses because one member vanishes. Catching a ghost is pointless if the lecturer drowns in alerts and never acts. The brief for Scenario 03 asks for skill-based formation, role assignment, capacity-aware tasking, conflict detection, and faculty governance, and those are not five features. They are one pipeline where the output of each stage is the input to the next.

Constraints we had to design inside

A few hard limits shaped every later decision, so we wrote them down before sketching any solution.

Constraint**What it forced on the design**

Scale	Formation has to run across an entire cohort at once, not team by team, so the matching had to be a batch process with predictable runtime rather than something a human nudges along.
Fairness floor	No team can be the dumping ground for the weakest students while another team gets every strong one. This had to be guaranteed by the maths, not left to a lecturer to eyeball afterwards.
Legal basis	Sri Lanka's PDPA and Computer Crimes Act apply. Any monitoring needs a lawful basis, consent, and a retention limit, which rules out collecting data quietly and deciding later why.
Human in the loop	An automated system should not be the final word on a student's standing. Every consequential output needs a faculty override and an appeals route.
Patchy data	Not every institution has a clean LMS feed, and not every student has prior grades. The system has to work fully without that data, treating it as a bonus when present.
Surveillance line	Monitoring contribution is reasonable. Covert tracking is not. The design had to deliver the detection without crossing into something students were never told about.

What we assumed about the scenario

Some of the brief is underspecified, so we made our assumptions explicit rather than letting them hide in the design. We assumed a faculty can realistically collect profile data through a web form, and that students have institutional Google accounts, since the workspace layer leans on that. We assumed a meaningful share of students will have no usable LMS history, so the skill model treats LMS as optional evidence and never depends on it. We assumed that if a survey is announced as a skill test, people game it, which is why the survey is presented as a profiling exercise and explained afterwards instead. We assumed engagement signals are approximate. A commit count is a proxy for contribution, not a measurement of it, so signals feed human judgement rather than replacing it. And we assumed that consent only counts if it comes before collection, which pushed every disclosure to the onboarding step.

One assumption is worth flagging on its own. We assumed that a balanced team is not automatically a functional team. Two people can be a perfect skill match and still not work well together. That gap between balanced and functional is exactly why the system keeps watching after formation rather than declaring victory once teams are published.

02. Proposed Solution

AEGIS is a background engine that does the heavy work quietly and surfaces only what a human needs to act on. It verifies skills before forming teams, screens proposals for duplicates, builds balanced teams, splits tasks by capacity, then keeps an eye on contribution for the rest of the semester. The lecturer never sees the engine. They see a dashboard of triaged decisions.

Who uses it, and what each role can do

Role	Responsibilities and limits
System Admin	Approves lecturer accounts before anyone gets access, sets up academic years, semesters, departments and modules, and assigns lecturers to specific cohorts. Has full visibility but is the only role that does setup. Lecturers cannot self-provision.
Lecturer / Supervisor	Sees only the students an admin assigned to them. Monitors team health, behavioural flags and contribution, creates and manages projects, overrides any AEGIS recommendation, and decides on appeals. Receives ready-to-send intervention messages instead of raw alert data.
Student	Self-registers and completes a structured onboarding profile, then is placed into a team by the engine. Sees their own adjusted skill profile and picks a role within the team that fits it, instead of being given a mismatched one. Can view their placement reason and appeal it.

How each of the four problems gets solved

Each failure mode from Section 1 maps to a specific mechanism rather than a vague promise.

Inflated skills, handled by evidence not by a binary check

Instead of accepting a self-report or hard-overriding it, AEGIS discounts every declared skill by a confidence factor based on how much evidence backs it. A confident 5 out of 5 with nothing behind it gets pulled down toward the middle. A 4 out of 5 that matches a verified grade is trusted at full weight. The score is triangulated across three channels so no single number carries the decision: an LMS grade sync where available, an adaptive survey framed as a profiling step, and an optional preferred-teammate signal that the engine may or may not honour. A week-2 calibration task then checks the profile against real work, and if observed throughput contradicts a claim later, the confidence factor slides down gradually rather than snapping.

Duplicate proposals, caught before teams lock

Every submitted abstract is turned into a term-frequency vector and compared against the others using cosine similarity. Any pair scoring at or above 0.75 is flagged for faculty review, with the option to make a group shift their stack or domain before the portal reopens. We chose a flag with an override rather than a hard block, because two proposals can look similar on text and still be genuinely different in practice.

Sympathy carry and ghosting, watched continuously

After teams start working, AEGIS polls workspace activity and runs three checks. If a high performer is completing work formally assigned to someone else past a 0.95 contribution ratio, submission approval halts and the carrier is prompted to re-delegate, which protects them from burnout and protects the individual grade record. A three-tier protocol handles the opposite case: a soft nudge when a footprint drops, task redistribution after six silent days, and a critical alert to faculty after ten. A team health score and a burnout flag run alongside, so an overloaded member shows up as clearly as a missing one.

Administrative overhead, removed by triage

The engine distils every anomaly into a ranked alert inbox so faculty never read raw telemetry. Each alert arrives with context, a set of options, and a pre-written, honest check-in message the lecturer can send as is. Every decision taken through the dashboard is logged with a timestamp and a reason, which gives the institution a clean evidence trail for grade disputes.

The supporting features

Around that core sit a set of smaller tools that make the experience usable rather than just correct. An AI project recommender suggests best-fit projects before allocation runs. A project demand predictor scores how oversubscribed each project is likely to be, so faculty can cap or duplicate popular ones early. A team compatibility score estimates whether a proposed group will actually function, based on schedule overlap and working style. A skill gap analyser surfaces missing competencies before a team is confirmed. A digital team contract pins down meeting frequency and responsibilities at formation. A contribution ledger logs every task, file and meeting into one normalised score that faculty can lean on during disputes. At project end, a portfolio generator turns the record into a clean summary a student can use for internships.

The thread running through all of it. AEGIS is built on one rule: full detection power, delivered with full transparency. Every behavioural risk a covert system could catch, this catches too, but students are told about it from day one. After placement they can see their own adjusted profile and the reasoning behind where they landed. That single decision, to detect openly rather than quietly, is what keeps the monitoring on the right side of the policy stack rather than turning it into surveillance.

03. Algorithmic and Logical Approach

This is the part that separates AEGIS from a basic matcher. What follows is the logic only, written as formulas, pseudocode and flow steps. No source code. The pipeline runs in three phases, and the order matters because each stage trusts the cleaned output of the one before it.

ENTRY	PHASE A	PHASE B	ASSIGN	PHASE C
Registration and consented onboarding	Triangulate inputs, verify skill by evidence	Cosine duplicate gate, then maximin formation with SPA	Student-led roles, capacity-based tasks, workspace provisioned	72h telemetry feeds the faculty oversight dashboard

The pipeline runs left to right. Each stage trusts the cleaned output of the one before it, which is why duplicate screening happens before formation and verification happens before either.

Phase A: verifying skill instead of trusting it

Each declared skill level L for a student is discounted by an evidence-confidence factor C before anything downstream uses it. This is a smooth discount, not a hard override. The adjusted score $\hat{A}$ and the per-discipline capability scalar S_{pillar} are defined as

$$\hat{A}(i, k) = L(i, k) C(i, k) \quad (1)$$

$$S_{\text{pillar}} = w_1 \text{LMS}_{\text{grade}} + w_2 \text{Survey}_{\text{score}} \quad (2)$$

C value	Basis for that confidence level
1.0	Verified institutional grade or assessed record
0.8	Portfolio or prior work submitted
0.6	Self-report only, the default, no LMS data required
0.5	Self-report later contradicted by observed task throughput

A confident but unsupported 5 lands near 2.5 to 3.0, while a verified 4 is trusted at 4.0. The method still works with no LMS feed, because the default $C = 0.6$ needs no external data. Each student compiles into a profile object holding the adjusted skill matrix, a capacity allowance in hours, project preferences and an optional teammate. No risk label is set at intake. Disengagement risk is computed only from live behaviour once the project starts.

Phase B: clean the proposals, then build fair teams

Before any team locks, proposal abstracts are vectorised and compared pairwise. Two proposals are flagged for faculty review when the cosine similarity of their term-frequency vectors reaches the threshold:

$$\text{Sim}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}, \quad \text{flag if } \text{Sim}(\mathbf{A}, \mathbf{B}) \geq 0.75 \quad (3)$$

Formation then runs the Abraham and Manlove Student-Project Allocation algorithm together with a maximin objective. The global goal is one line, and it is the reason no team can be stacked with the weakest students:

$$\max \min_T \text{TeamScore}(T) \quad \text{subject to all hard constraints} \quad (4)$$

Maximising the score of the weakest team makes a doomed group impossible to assemble. An anti-concentration rule caps any single specialisation at roughly 60 percent of a team so no project becomes a sandbox of one skill. Oversubscription is resolved by a deferred-acceptance cascade in which a contested student is ranked by a priority score that rewards filling a scarce critical skill:

$$\text{Priority}(i, p) = \text{Fit}(i, p) + \text{RareSkillBonus}(i, p), \quad \text{RareSkillBonus} = +15 \quad (5)$$

Algorithm 1 Oversubscription resolution (SPA, many-to-one)

```

while some student is free and has an untried preference:
    p ← that student's next preferred project
    provisionally place the student in p
    if p is over capacity:
        rank contested students by Priority (Eq. 5)
        keep the top cap(p) students, release the rest
return a stable, envy-reduced assignment of students to projects

```

The formation loop seeds rare critical skills first, expands greedily on coverage and schedule overlap, then runs a local-search swap pass. A swap is accepted only if it raises the minimum team score and breaks no hard rule. Anything that cannot be placed cleanly goes to a faculty-reviewed exception pool instead of being forced.

Team score component (weight)	Meaning	Defends against
Critical-skill coverage (30)	Fraction of critical skills met using $\hat{A}$	A team with a missing skill
Schedule compatibility (20)	Common hours over required, capped	A team that can never meet
Role coverage (15)	Average role-fit of filled roles	Weak role fit
Preference satisfaction (15)	1st choice 1.0, 2nd 0.7, 3rd 0.4	Low morale
Workload balance (10)	Evenness of member capacity	Uneven load
Experience balance (10)	Closeness to the cohort skill target	The sandbox effect

Roles and tasks

Roles are not forced. AEGIS works out each member's skills independently and shows every student their own evidence-adjusted profile, so each can see their real strengths. The team then assigns roles from that shared picture, so a person takes a role that fits their skills rather than one that does not. An empty role or flagged mismatch still notifies the lecturer, and where a suggestion is wanted, role fit is scored across skill, preference, experience, availability and a small leadership weight. Tasks are then handed out in proportion to declared capacity, which stops overload at assignment time rather than after:

$$\text{Task_Share}(i) = \frac{\text{cap}(i)}{\sum_{j \in \mathcal{T}} \text{cap}(j)} \quad (6)$$

$$\text{Hours}(i) = \text{Task_Share}(i) \times \text{Total_Project_Hours} \quad (7)$$

$$U(i) = \frac{\text{AssignedEffort}}{\text{AvailableHours} - \text{ExistingCommitments}} \quad (8)$$

The target utilisation band is $0.7 \leq U(i) \leq 1.0$ per student. Once $U(i) > 1.2$, automatic rebalancing triggers, and a single member far above the team median is the early tripwire for a sympathy carry. Large milestone tasks are split into subtasks before assignment so work routes to the right person instead of landing whole on one desk.

Phase C: monitoring without surveillance

After provisioning, AEGIS polls workspace APIs every 72 hours and rolls the signals into one health score per team, recomputed on the same cycle. The five normalised rates are task completion r_{tc} , attendance r_{at} , workload balance r_{wb} , engagement r_{en} and milestone progress r_{mp} :

$$\text{Health} = 100 (0.30 r_{tc} + 0.15 r_{at} + 0.25 r_{wb} + 0.15 r_{en} + 0.15 r_{mp}) \quad (9)$$

Teams classify as healthy at 75 and above, at risk between 50 and 74, and critical below 50. On top of the score, two behavioural checks run: a poached-task intercept when one student does another's assigned work past a 0.95 contribution ratio, and a burnout flag when a member's utilisation runs at twice the team average:

$$U(A) \geq 2 \overline{U}_{\text{team}} \implies \text{sympathy-carry burnout warning}$$

(10)

Algorithm 2 Poached-task intercept (per 72h poll)

```
if Task_Assigned(B) and Commit_Author(A) and Contribution_Ratio(A) ≥ 0.95:
    halt submission approval
    log a workload attribution warning
    prompt the carrier (A) to re-delegate the work
```

Tier	Trigger	Automated response
1	Active footprint drops 40 percent or more	Soft check-in nudge sent to the student
2	Six consecutive days of zero input	Non-critical tasks unassigned and redistributed; faculty notified
3	Ten consecutive days of zero input	Critical alert routed to the faculty overwatch console

Hard constraints are checked before any scoring: every eligible student placed once, team size in range, critical skills covered or formally flagged, nobody assigned past capacity, no serious timetable clash. Soft goals like preference satisfaction and balanced skill spread are optimised only after the hard rules pass, and a high soft score can never override a hard breach. Ties break on the higher minimum team score first, then critical-skill coverage, then preference, with a transparent random draw as the last resort.

SECTION FOUR

04. Feasibility and Scalability

A clever algorithm that needs data nobody has, or that falls over at cohort scale, is not a solution. We checked AEGIS against both: can a faculty run it with the data they actually hold, and does it hold up as the numbers grow.

Operational feasibility

The system uses data a faculty can collect through a web form, so it can start as a form and a database and grow from there. It removes the repetitive part of allocation while keeping a human responsible for the exceptional decisions, which is the part people are good at and software is not. Nothing in the first phase needs an LMS, a GitHub integration or live telemetry, so an institution can adopt it in stages.

Phase	Scope	Depends on
Phase 1	Profiles, project preferences, evidence-weighted scoring, matching, basic team formation, faculty review.	A web form and a database. Nothing else.
Phase 2	Automated role matching, conflict detection, task allocation, the severity dashboard.	Supervisor task data.
Phase 3	Consented workload monitoring, supervisor analytics, appeals, semester-over-semester tuning.	LMS, repository or chat integrations where available.

Technical feasibility and behaviour under load

The data model is ordinary relational records, and the scoring is transparent arithmetic rather than an opaque model, so a result can always be explained to a student who asks. The pipeline is modular: profile collection, matching, formation, role assignment, task allocation and monitoring are separable stages, which lets the system scale by stage instead of as one monolith.

Independent project pools are processed in parallel, repeated compatibility calculations are cached, and a mid-semester change recomputes only the affected team and never reshuffles the rest of the cohort. Notifications run through a queue so a spike in activity does not stall the matcher. On the workspace side, a 72-hour polling interval across thousands of teams stays well inside the Google Drive API budget of 12,000 requests per 100 seconds, and cohort-wide file provisioning is batched into seconds.

Where it holds up

- Batch formation for the first run, event-based updates afterwards.
- Deterministic, re-runnable allocation with the same seed, which makes audits and recovery straightforward.
- Incremental recompute, so one dropout costs one team, not the cohort.

What it deliberately does not do

- It does not read document contents, only activity metadata.
- It does not score sensitive attributes like ethnicity, religion or gender.
- It does not let an automated decision stand without a human review path.

Known limitations, and what we do about each

We would rather name the weak points than hide them. None have perfect fixes, so each is paired with a mitigation that reduces the damage rather than a claim that it is solved.

Limitation	Mitigation
Students may still exaggerate skills.	Evidence weighting, the week-2 validation task, lecturer confirmation, and throughput feeding back into the confidence factor.
A balanced team may not be socially compatible.	A short review window, supervisor feedback, and carefully limited preference data.
Perfect allocation is sometimes impossible.	Severity-based flags and a faculty-reviewed exception pool for the unavoidable conflicts.
Scoring weights can introduce bias.	Publish the factors, compare team distributions through a bias audit, and review the weights every semester.
Competing fairness definitions cannot all hold at once.	State the chosen one, maximin, in the open, and keep a human override for the cases it does not serve.

How we would know it worked

The targets below are goals, not results we are claiming. We would judge a deployment on whether every eligible student ends up assigned or flagged, whether most teams cover their mandatory skills unaided, and whether manual allocation time drops sharply. Over a semester we would watch the skill gap between the strongest and weakest team, workload variance, and the number of overrides and appeals. That last one matters most: a healthy appeals count is not zero. It shows students trust the route enough to use it, and that faculty are reviewing rather than rubber-stamping the engine.

Where this leaves us. AEGIS is not magic, and we have not tried to sell it as such. It replaces the parts of allocation a computer does well, the verifying, matching and watching, and hands back the parts a human does well, the judging and deciding. The monitoring earns its keep because it is open about itself; the matching earns its keep because a person can override it. Take either away and it stops being the system we designed.